Midnight to Dawn — Technical Design Document

Nov 8, 2026 · @Akhilesh

Overview

We are building a single-page, scroll-driven 3D portfolio for Akhilesh Singh that plays the *Midnight to Dawn* screenplay as one continuous camera journey across 8 acts, with a fast plain-HTML version for recruiters and search engines. This document is written so Claude Code can build it milestone by milestone; the screenplay doc remains the source of truth for what each shot looks like.

Goals

1. **Cinematic:** every act transition is continuous, with no page reloads and no visible loading between acts.
2. **Efficient:** runs at 60 fps on a mid-range laptop and 30+ fps on a mid-range Android phone.
3. **Content-driven:** all text (skills, projects, experience, contact) lives in typed data files, so updating the portfolio never means touching 3D code.
4. **Recruiter-safe:** name and role visible within 3 seconds, every act skippable, a plain version always one click away.

Success targets

Metric	Target
Frame rate, desktop (mid-range GPU)	60 fps steady
Frame rate, mobile (mid-range Android)	30 fps minimum on the Lite tier
Download before the loading screen appears	1.5 MB or less
Time to loading screen visible	Under 1.5 s on 4G
Lighthouse Performance, recruiter page	95 or higher
Lighthouse Accessibility, recruiter page	95 or higher

Metric	Target
Full experience runtime	About 4 min watched fully, under 60 s via quick menu

In scope

Acts 0–7 from the screenplay as fully 3D scenes (rigged characters, environments, materials, cameras) authored in Blender with Claude Code, the recruiter page, the contact form with email delivery, desktop and mobile support, sound, and deployment.

Out of scope for version 1

Multiplayer or visitor accounts, a CMS, and multiple languages.

Tech stack

Every asset, character and camera move is authored in Blender first; Next.js + React Three Fiber then plays those scenes in the browser, GSAP drives every timed or scrolled animation, and Zustand is the single shared state. Install the latest stable version of each at setup and pin exact versions in `package.json`; do not upgrade major versions mid-build.

Authoring (Blender side)

Layer	Tool	Job in this project	Why this one
3D authoring	Blender (latest stable)	Model, rig, animate, shade, light, bake and lay out every scene and camera	Each page's full scene is built and approved in Blender before any web code
Claude Code ↔ Blender	A Blender MCP server for live control, plus saved `bpy` scripts run with `blender --background --python`	Live building while you watch, then reproducible rebuilds	Scripts make every scene rebuildable and reviewable in git

Layer	Tool	Job in this project	Why this one
Rigging	Rigify (built into Blender)	Humanoid skeletons for the swordsman, Pillars, demons, King	One shared rig layout lets animations be reused across characters
Animation	Blender actions, exported as glTF animation clips	Falls, idles, breathing, strikes, transformations	Played in the browser with three.js `AnimationMixer`
Crowds	Vertex animation textures (VAT), baked by script	Up to 200 animated weak demons	One draw call for the whole crowd; skinned meshes can't scale that far
Source control for art	Git LFS	Stores `.blend` files and raw textures	Keeps the repository fast

Runtime (browser side)

Layer	Library	Job in this project	Why this one
Framework	Next.js (App Router) + TypeScript strict	Routes, the static recruiter page, the contact API route, font loading	One project covers the 3D experience, SEO page and backend
3D engine	three.js	Rendering, skinned meshes, animation mixer	The standard WebGL engine, huge ecosystem
3D in React	@react-three/fiber	Scene graph as React components, frame loop	Clean component structure per act
3D helpers	@react-three/drei	GLTF/KTX2 loaders, `useAnimations`, `Html` (HTML placed on 3D surfaces), `Detailed` (LOD), `PerformanceMonitor`	Saves weeks of boilerplate

Layer	Library	Job in this project	Why this one
Anime look	Custom toon (cel) shader + inverted-hull outlines	Flat anime shading with ink outlines, matched to the Blender materials	Strong style at low GPU cost
Post effects	@react-three/postprocessing	Bloom, vignette, motion blur, screen split, dawn grade	Merges effects into few passes, fast
Animation	GSAP + ScrollTrigger	Shot timelines, scroll scrubbing, uniform tweens	Best timeline control on the web; all plugins are free
Smooth scroll	Lenis	Smooth, controllable scroll; can be locked during cinematic beats	Works with ScrollTrigger out of the box
State	Zustand	Current act, time scale, quality tier, mute, gates	Tiny, readable from inside the render loop without re-renders
Audio	Howler.js	Sound effects, music, audio sprites	Handles browser audio unlock and formats
Device check	detect-gpu	Picks the starting quality tier	Benchmark-based GPU scoring
Validation	zod	Content data and contact form validation	One schema for types and runtime checks
Email	Resend	Delivers contact messages	Simple API, free tier
Styling (DOM)	Tailwind CSS	Text placed on 3D surfaces, menus, recruiter page	Fast to build, small CSS

Layer	Library	Job in this project	Why this one
Asset tools	gltf-transform CLI, KTX-Software	Compress models (meshopt) and textures (KTX2)	Smallest file sizes for the web
Dev only	Leva, r3f-perf	Live-tweak values; FPS, draw-call and triangle overlay	Tuning without code edits
Testing	Vitest, Playwright, Lighthouse CI	Unit, end-to-end and performance checks	Runs locally and in CI
Hosting	Vercel	Deploys, preview links per branch	Built for Next.js

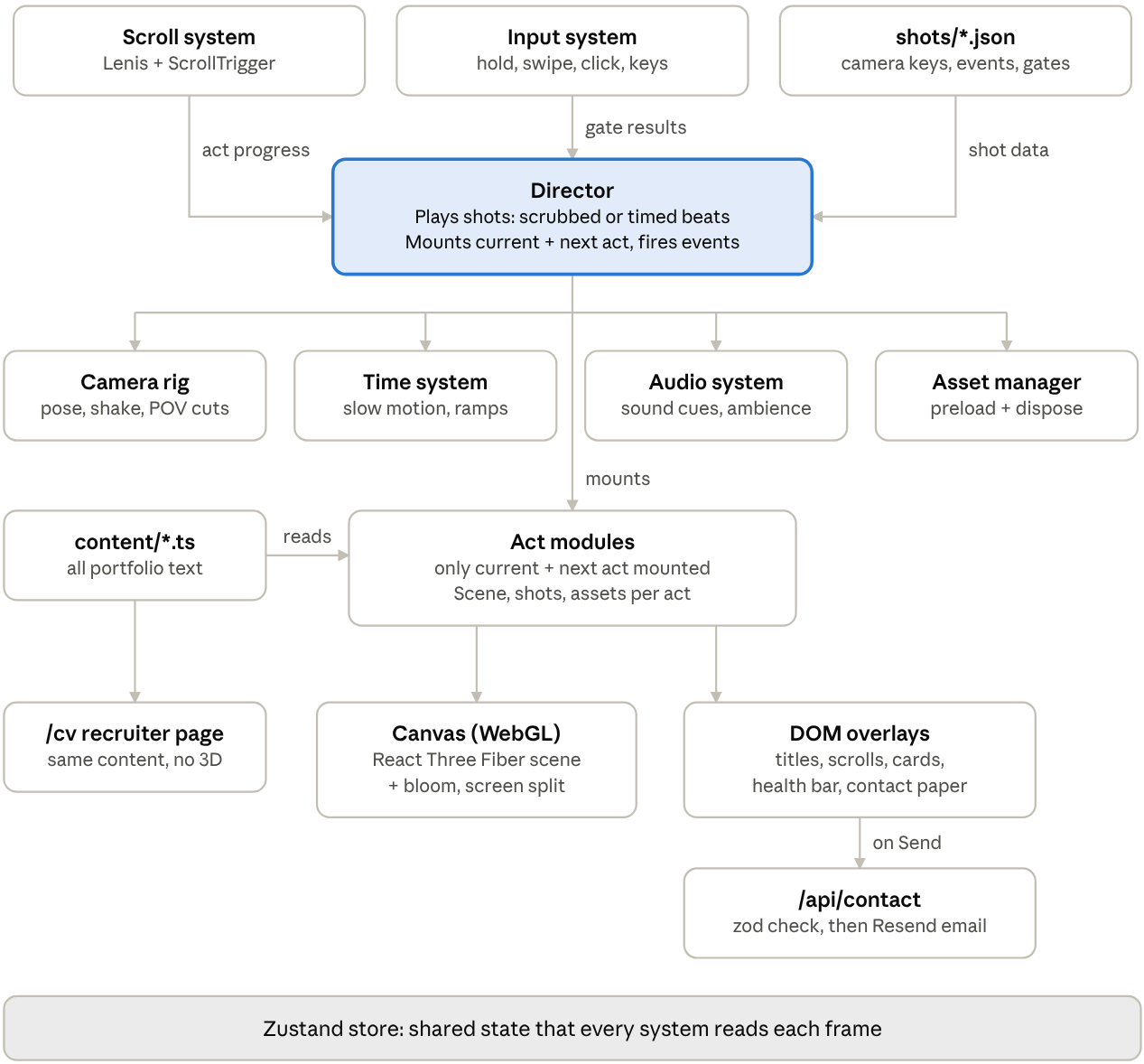
Rule for Claude Code: do not add any library not listed here without asking first. Every extra dependency adds download weight.

Architecture

Scroll and input flow into one Director, which reads shot data, drives the camera, time, audio and asset systems, and mounts at most two acts; the acts draw into one WebGL canvas plus HTML overlays.

The Director turns scroll and input into every shot

Data flows down; content and shot files are the only inputs you edit



system architecture · Director at the center

Read it top to bottom: the only files you edit to change the story are `shots/*.json` (camera work) and `content/*.ts` (portfolio text), and the recruiter page reads the same content without any 3D.

Project structure

The code is split so that 3D source, content, shot data, systems and acts never mix: you build scenes in `blender/`, edit `content/` to change the portfolio text, and edit `acts/` to change how a scene plays.

```
portfolio/
├─ CLAUDE.md                    # rules for Claude Code (see Build plan)
├─ blender/                     # SOURCE OF TRUTH for all 3D (Git LFS)
|  ├─ library/                  # shared: toon material, outline, kit
pieces, props
|  ├─ characters/               # swordsman, pillars/, demons/, king, crow,
koi
|  ├─ scenes/                   # one .blend per act: act1-fall.blend ...
|  ├─ scripts/                  # bpy scripts Claude Code writes and re-runs
|  |  ├─ build_<act>.py         # builds or updates a scene from scratch
|  |  ├─ export_glb.py          # exports collections to raw-assets/models
|  |  ├─ export_shots.py        # camera keyframes -> shots/<act>.json
|  |  └─ bake_vat.py            # crowd animation -> vertex animation
textures
|  └─ renders/                  # approved preview renders per act
├─ raw-assets/                  # exported, not yet optimized (not deployed)
├─ content/                     # ALL portfolio text, typed + zod-validated
|  ├─ profile.ts                # name, role, about scrolls, portrait
|  ├─ skills.ts                 # skill groups mapped to Pillars
|  ├─ projects.ts               # projects mapped to Upper Rank demons
|  ├─ experience.ts             # boss phases = timeline entries
|  └─ contact.ts                # links + resume file
├─ shots/                       # camera + event data per act (generated +
edited)
|  └─ act1-fall.json ...
├─ public/
|  ├─ models/                   # *.glb (meshopt), characters with animation
clips
|  ├─ textures/                 # *.ktx2
|  ├─ vat/                      # vertex animation textures for crowds
|  ├─ audio/                    # *.webm (Opus) + *.mp3 fallback
|  └─ resume.pdf
├─ src/
|  ├─ app/
|  |  ├─ layout.tsx
|  |  ├─ page.tsx               # loads <Experience/> client-only
|  |  ├─ cv/page.tsx            # recruiter mode, static HTML
```

```
|  |  └─ api/contact/route.ts    # contact form endpoint
|  ├─ experience/
|  |  ├─ Experience.tsx          # <Canvas>, Director, HUD controls
|  |  ├─ director/               # Director, act registry, gates
|  |  ├─ systems/                # camera, time, scroll, input, audio,
assets, quality
|  |  ├─ characters/             # Character.tsx (GLB + clips + LOD), useClip
hooks
|  |  ├─ acts/
|  |  |  ├─ act1-fall/
|  |  |  |  ├─ index.ts           # ActModule export (see Core systems)
|  |  |  |  ├─ Scene.tsx          # loads the act's GLB, wires clips and VFX
|  |  |  |  └─ assets.ts          # list of files this act needs
|  |  |  └─ ...                    # act0 to act7, same shape
|  |  ├─ materials/              # ToonMaterial, OutlineMaterial (match
Blender)
|  |  ├─ vfx/                    # ink, ash, particles, lanterns, screen
split, VAT crowd
|  |  ├─ shaders/                # shaders as .glsl.ts string modules
|  |  └─ ui/                     # HUD only: QuickMenu, SkipButton, Mute,
BreathRing
|  ├─ store/useStore.ts          # Zustand store
|  └─ lib/schemas.ts             # zod schemas shared by content + API
├─ scripts/optimize-assets.mjs   # gltf-transform + KTX2 batch job
└─ tests/
   ├─ unit/
   └─ e2e/
```

Naming rules: components in PascalCase, hook files start with `use`, one act per folder, no act imports another act's files (shared things go in `systems/`, `characters/`, `materials/`, `vfx/` or `ui/`). Blender collections are named `act<N>_<thing>`, animation actions `<character>_<move>`, and camera markers use the screenplay shot ids.

Core systems

The whole site runs on one idea: the **Director** plays each act as a list of shots, where a shot is either *scrubbed* by scroll or played as a timed *beat*, and interactive moments are *gates* that pause the story until the visitor acts. Every system below serves the Director.

1. Director

The Director owns the story order and is the only thing allowed to change the current act.

- Keeps an ordered registry of 8 `ActModule`s.
- Gives each act a scroll length (in screen heights); the page height is the sum, and one ScrollTrigger per act turns scroll into that act's progress, 0 to 1.
- **Mounts at most 2 acts at once:** the current act and the next. When the current act passes 50% progress, the next act's assets start loading. After a transition finishes, the previous act is unmounted and its GPU memory disposed.
- Reads `events` from shot data at the right time and sends them to audio, UI (title cards) and time systems.
- Handles **jumps** from the quick menu: an ink-wipe covers the screen for about 0.6 s, the target act mounts at its start, the wipe clears. If the target isn't loaded, the wipe shows a small breathing ring until it is.

```
type ActId = 'act0' | 'act1' | 'act2' | 'act3' | 'act4' | 'act5' | 'act6' |
'act7';

interface ActModule {
  id: ActId;
  menuLabel: string;             // "Home", "About", "Skills"...
  shots: Shot[];                 // loaded from shots/<act>.json
  Scene: React.ComponentType;    // the act's 3D content
  assets: AssetManifest;         // files to preload
  onEnter?: () => void;
  onExit?: () => void;
}
```

2. Shot data format

Shots are **data, not code**, so camera work can be tuned without touching components. Each act's shots live in `shots/<act>.json`.

```
type Vec3 = [number, number, number];

interface CameraKey {
  t: number;          // 0..1 within the shot
  pos: Vec3;
  target: Vec3;       // point the camera looks at
  fov?: number;
```

```
  roll?: number;      // degrees, for upside-down and Dutch angles
  ease?: string;      // GSAP ease name
}

interface ShotEvent {
  t: number;          // 0..1 within the shot
  type: 'sound' | 'title' | 'timeScale' | 'shake' | 'gate' | 'custom';
  payload: Record<string, unknown>;  // e.g. { cue: 'taiko' } or { to: 0.05,
ramp: 0.3 }
}

interface Shot {
  id: string;               // "1.7-name-reveal" (matches screenplay
numbering)
  mode: 'scrub' | 'beat';   // scrub = follows scroll, beat = plays in real
time
  scrollLength?: number;    // scrub only, in screen heights
  duration?: number;        // beat only, in seconds
  camera: 'main' | 'pov';
  keys: CameraKey[];
  events: ShotEvent[];
}

interface Gate {
  id: string;
  kind: 'hold' | 'swipe' | 'click' | 'read';
  holdMs?: number;                  // hold gates, e.g. 3000 for the Final
Form
  autoResolveMs: number | null;     // null = visitor must act
}
```

Beat behavior: when scroll reaches a beat, scrolling is locked (`lenis.stop()`), the beat plays, then scrolling unlocks. A visitor who scrolls hard during a beat makes it play at 2× speed instead of being ignored.

Cameras are authored in Blender. Each act's `.blend` has one animated camera; timeline markers named with screenplay shot ids (`1.7-name-reveal`) split it into shots, and custom properties on each marker set `mode`, `scrollLength` or `duration`. `blender/scripts/export_shots.py` samples the camera at every keyframe and writes `shots/<act>.json` in the format above, so the browser plays exactly the framing approved in Blender. Events and gates are added in the JSON after export and preserved on re-export (the script merges by shot id).

3. Camera rig

- One main `PerspectiveCamera` plus a POV camera attached to the swordsman's head bone. A cut = switching the active camera.
- Position between keys uses a smooth curve (`CatmullRomCurve3`); target, fov and roll are interpolated with the key's ease.
- **Shake** is added on top of the final pose using noise, with an amplitude that fades out. Mouse parallax (a few degrees) is also added on top, never baked in.
- **Never** move the camera through React state. All camera changes happen by changing refs inside `useFrame`, which avoids re-rendering React 60 times a second.

4. Time system (slow motion)

- The store holds one `timeScale` value (1 = normal, 0.05 = deep slow motion).
- A shared hook `useActFrame((state, scaledDelta) => ...)` passes `delta × timeScale` to every animated thing: particles, ink, ash, cloth sway, characters.
- `setTimeScale(to, rampSeconds)` tweens the value, which produces speed ramps for free. `gsap.globalTimeline.timeScale()` is kept in sync for beat timelines.
- Scrub shots are not slowed by time scale (scroll stays in control); only the things *inside* the scene slow down.

5. Scroll system

- Lenis provides smooth scrolling and drives ScrollTrigger: `lenis.on('scroll', ScrollTrigger.update)` and GSAP's ticker calls `lenis.raf`.
- `lockScroll()` / `unlockScroll()` wrap `lenis.stop()` / `lenis.start()` for beats and gates.
- Keyboard: Space and Arrow Down advance; Arrow Up goes back within an act.

6. Input system

Gesture	How it's detected	Keyboard alternative
Hold (breathing)	Time between `pointerdown` and `pointerup`; releasing early drains the ring	Hold Space

Gesture	How it's detected	Keyboard alternative
Swipe (slash)	Pointer path recorded for at most 400 ms, at least 25% of screen width long; emits start and end points	Press Enter for an automatic diagonal slash
Click	Raycast against interactive objects (scrolls, beacons, doors)	Tab to focus, Enter to activate
Read	Gate resolves when the panel is closed or after a timer	Escape closes panels

7. Audio system

- Howler.js with **audio sprites**: one file holds all short effects (taiko, clacks, whooshes, inhale, chimes), so it's one download instead of 30.
- One ambience loop per act, crossfaded over 1.5 s on act change.
- Shot events trigger cues by name: `{ type: 'sound', payload: { cue: 'taiko-double' } }`.
- Audio unlocks on the Enter click. Mute state is saved in `localStorage`.

8. Asset manager

- Each act lists its files in `assets.ts`. The manager preloads them (drei's `useGLTF.preload`, KTX2 loader, Howler) and reports progress to the store.
- The loading screen waits only for Act 1's assets; everything else loads in the background, one act ahead.
- On unmount, an act disposes its geometries, materials and textures.

9. Quality system

- At startup, detect-gpu chooses a tier: **High**, **Medium** or **Lite**.
- During play, drei's `PerformanceMonitor` drops one tier if frame rate stays low for 3 seconds. The tier only changes between shots, never mid-shot, so nothing visibly pops.
- What each tier changes is listed under Performance.

10. Store (Zustand)

```
interface AppState {
  actId: ActId;
  actProgress: number;                        // 0..1
  timeScale: number;
  quality: 'high' | 'medium' | 'lite';
  muted: boolean;
  reducedMotion: boolean;
  gates: Record<string, 'pending' | 'done'>;
  visitedPillars: string[];
  defeatedRanks: string[];
  // actions
  goToAct: (id: ActId) => void;
  setTimeScale: (to: number, rampSeconds?: number) => void;
  resolveGate: (id: string) => void;
}
```

Components inside the render loop read the store with `useStore.getState()` (no re-render); DOM overlays use normal selectors.

Content data model

All portfolio text lives in `content/*.ts` files that follow these types; both the 3D experience and the recruiter page read the same data, so they can never disagree. Each type has a matching zod schema in `src/lib/schemas.ts`, and the build fails if content doesn't match.

```
// content/profile.ts
interface Profile {
  name: string;                // "Akhilesh Singh" — written on the door
  role: string;                // "AI Engineer · Full-Stack Developer"
  tagline: string;             // one line for loading screen + SEO
  portrait?: string;           // path to ink-style portrait image
  scrolls: AboutScroll[];      // exactly 4, shown in the Quiet Room
}
interface AboutScroll {
  id: 'who' | 'path' | 'drive' | 'beyond';
  label: string;               // brush word on the outside
  body: string;                // markdown, max ~120 words
}
```

```
// content/skills.ts
type Pillar = 'serpent' | 'mist' | 'water' | 'flame' | 'sound'
            | 'stone' | 'wind' | 'insect' | 'love';
interface SkillGroup {
  pillar: Pillar;
  title: string;               // "AI and LLMs"
  featured: boolean;           // true = full sequence, false = background
only
  formName: string;            // name called before the strike
  skills: Skill[];
}
interface Skill {
  name: string;
  icon: string;                // SVG path in /public/icons
  years?: number;
  usedIn: string[];            // project ids
}

// content/projects.ts
interface Project {
  id: string;
  rank: 1 | 2 | 3 | 4 | 5 | 6;  // 1 = best project, fought last
  demonName: string;           // "The Endless Inbox" (the problem)
  demonPower: 'letters' | 'threads' | 'scales' | 'gears' | 'masks' |
'charts';
  helperPillar: Pillar;        // Pillar who fights beside the swordsman
  title: string;
  pitch: string;               // one line
  problem: string;
  solution: string;
  result: string;              // numbers where possible
  stack: string[];             // skill names, must exist in skills.ts
  previewVideo?: string;       // short muted .webm loop
  links: { demo?: string; github?: string; caseStudy?: string };
}

// content/experience.ts
interface ExperiencePhase {
  id: string;
  role: string;
  org: string;
  start: string;               // ISO date "2026-06-01"
  end: string | 'present';
```

```
  bossForm: 'arms' | 'whips' | 'armor' | 'spikes' | 'final';
  achievements: string[];       // 2–3 lines written in ink
}

// content/contact.ts
interface Contact {
  email: string;
  linkedin: string;
  github: string;
  resume: string;               // "/resume.pdf"
}
```

Validation rules the build enforces: exactly 4 scrolls; 5–6 featured skill groups; at most 6 projects with unique ranks; every `stack` entry and `usedIn` id must exist; experience phases sorted oldest first.

Scene specs per act

Every act is a Blender scene exported as GLB, with real rigged 3D characters, kit-built environments and in-world VFX. Text people must read (scroll contents, project cards, health bar labels, the contact form) is printed **on 3D objects** — the scroll, the talisman, the paper — using drei's `<Html transform occlude>`, which places real HTML on the object's surface: it moves, tilts and hides behind things like part of the world, while staying sharp, selectable and readable by screen readers.

Act	Scroll length	3D content (from Blender)	Key techniques	Asset budget
0 Loading	none	Ink ring + 3D sliding door, tiny scene	three.js core only, one shader	300 KB
1 Fall	6 screens	Swordsman (rigged), corridor kit, 300 lanterns, door tunnel	Infinite shaft by recycling kit pieces, instancing, ink written on the door in 3D	8 MB
2 Quiet Room	3 screens + read gates	Baked room, 4 rigged scrolls, wisteria, tea set	Baked lighting, scroll-unroll animation, text on scroll surfaces, tatami flip	6 MB

Act	Scroll length	3D content (from Blender)	Key techniques	Asset budget
3 Battlefield	10 screens	5–6 Pillars (shared rig), VAT demon crowd, platforms	Shared animation set, element VFX on sword bones, ash-to-icon particles	14 MB
4 Upper Ranks	6 screens (6 beats + swipe gates)	6 rigged Upper Rank demons, balconies	Power presets, screen-split pass, talisman cards with HTML on surface	12 MB
5 Demon King	6 screens + final hold gate	King with form variants, arena, debris kit	Shape keys + part toggles per form, 3D health-bar scroll, sunrise	10 MB
6 Dawn	1 screen	Act 7 scene behind post effects	White-out grade, eyelid vignette shader, focus pull	50 KB
7 Mansion	3 screens + form	Baked mansion + garden, rigged crow, VAT koi	Paper form on the 3D paper, crow flight clip along a curve	7 MB

Each act's assets stream in while the visitor is in the previous act, so these budgets add up over the visit but never block it.

Act 0 — Breath

- A tiny WebGL scene: the ink ring is the InkRing shader on a plane, the door is a small GLB whose panels slide on click. Only three.js core and this scene load first; everything else waits.
- Act 1's assets load behind it; the ring's fill shows their progress.

Act 1 — The Fall

- **Swordsman:** rigged GLB with clips `fall_loop`, `twist_dodge`, `draw_sword`, `falling_stroke`, `land_kneel`. Cloth and hair motion are baked into the clips in Blender.

- **Infinite shaft:** 6 kit pieces from Blender, 20–30 copies in a column; pieces passed by the camera move from above to below, so the fall is endless with fixed memory.
- **Lanterns:** one instanced mesh from the Blender lantern model, 300 on High; drift and sway in the shader; the red wave in shot 1.4 is one uniform with a height-based delay.
- **Door tunnel:** 12 recycled door GLBs; the two panels are separate objects animated to the clack cues.
- **Name on the door (1.7):** the name is written on the door's paper in 3D. Blender (or a script) creates a *stroke-order texture* where each brush stroke's pixels store when that stroke is drawn; the door shader reveals ink as one uniform goes 0 → 1, so it appears stroke by stroke on the actual door. On the dodge, the ink lifts off as GPU particles sampled from that texture and settles into the HUD logo.
- **Landing (1.9):** InkRibbon spiral attached to the sword-tip bone during `falling_stroke`.

Act 2 — The Quiet Room

- **Room:** one baked GLB, lighting baked in Cycles; 1–2 real lights only for the swordsman.
- **Scrolls:** each scroll is rigged with a few bones (or a shape key) so it unrolls in Blender; the web plays the clip. The text is HTML placed on the scroll's surface with `<Html transform occlude>` and fades in as it unrolls.
- **Tea ripple:** RingRipple shader on the tea surface, triggered by drum cues.
- **Tatami flip:** each mat a separate object with its hinge as origin; flip animation with stagger, authored in Blender.

Act 3 — The Battlefield

- **Pillars:** all use the same Rigify layout, so one animation set (`idle_combat` , `inhale` , `strike_wide` , `strike_thrust` , `land`) is shared and retargeted; each Pillar adds 1–2 signature clips.
- **Weak demons:** 1–3 crowd clips baked by `bake_vat.py` into vertex animation textures; one instanced mesh draws all 200, each with its own animation offset and dissolve value.
- **Element VFX:** one `ElementTrail` attached to the sword bone and one `ElementBurst` effect; each Pillar is a config (color, noise, shape texture).
- **Ash to icons:** ParticleMorph from the dissolving crowd to 3D skill icon meshes (extruded from SVG logos in Blender) that float in an arc. Hover shows an HTML card attached to the icon.

Act 4 — The Upper Ranks

- **Demons:** 6 rigged GLBs with `reveal`, `attack`, `hit`, `death` clips; extra limbs are extra bone chains.
- **Demon powers:** 6 presets (letters, threads, scales, gears, masks, charts), each an instanced set of small Blender meshes.
- **Swipe slash:** the input line drives the ScreenSplit pass and an InkRibbon slash in world space.
- **Project card:** a 3D paper talisman; its content (title, text, stack icons, video, links) is HTML on the talisman's surface.

Act 5 — The Demon King

- **King:** one rigged GLB; each `bossForm` is a mix of shape keys (body changes) and visibility toggles for extra parts (arms, whips, armor), with a clip per transformation.
- **Health bar:** a 3D hanging scroll that rides at the top of the frame (parented to the camera), with segment labels as HTML on its surface.
- **Light cracks:** emissive crack meshes that grow via a reveal uniform, plus light cones; bloom does the glow.
- **Final Form:** InkRing drawn in world space around the swordsman, then the SunriseGrade.
- **Disintegration:** 150–300 instanced debris pieces from the castle kit, tweened upward with spin. No physics engine.

Act 6 — Dawn

No new geometry: Act 7 mounts behind a full white grade; an eyelid vignette shader closes and opens twice while a depth-of-field focus pull goes from blurry to sharp.

Act 7 — The Mansion

- **Room and garden:** baked GLB; wisteria and grass as instanced foliage meshes with the WindSway shader; koi as a small VAT school.
- **Paper form:** real HTML inputs placed on the 3D paper's surface, so autofill, screen readers and phone keyboards all work.
- **Crow:** rigged GLB with `land`, `pick_up`, `fly` clips, following a curve out over the garden; the camera crane-up follows it to the end card, which is ink text drawn in the sky with the same stroke-reveal shader as the door.

Shaders and VFX catalog

The whole film needs 16 effects, each built once and reused through uniforms; no act gets a one-off effect without adding it to this table first. The toon and outline materials are built in Blender and in the browser with the same parameters, so what you approve in Blender is what ships.

Effect	What it is	Main uniforms / inputs	Used in	Lite tier
ToonMaterial	Cel shading: 2–3 hard light bands, rim light, works with skinning and VAT	`uBaseColor`, `uShadeColor`, `uBands`, `uRim`	Every character and prop	2 bands, no rim
InkOutline	Inverted-hull outline with brush-like width variation	`uWidth`, `uColor`, `uNoise`	Characters, key props	Characters only, no noise
VATCrowd	Instanced mesh animated by vertex animation textures	`uTime`, `uClip`, per-instance offset and dissolve	Weak demons, koi	60 instead of 200
StrokeReveal	Ink appears stroke by stroke from a stroke-order texture	`uReveal`, stroke-order texture	Name on the door, end card in the sky	Same
InkRibbon	Ribbon or tube mesh with noisy, bleeding ink edges	`uProgress`, `uColor`, `uNoiseScale`, `uTime`	Acts 1, 3, 4, 5	Fewer segments, no edge noise
InkRing	Brush-textured circle drawn by a fill value	`uFill`, `uColor`, `uGlow`	Act 0 loading, Act 5 Final Form	Same

Effect	What it is	Main uniforms / inputs	Used in	Lite tier
Dissolve	Object crumbles away by a noise threshold, glowing edge	`uDissolve`, `uEdgeColor`	Demons in Acts 3, 4, 5	Same (cheap)
ParticleMorph	GPU points flying from start shape to end shape	attributes `aStart`, `aEnd`, `aRandom`; `uProgress`	Name to logo, ash to icons, ash to talisman	40% particle count
LanternField	Instanced lanterns with drift, sway and a color wave	`uTime`, `uWaveHeight`, `uWaveColor`	Act 1	120 instead of 300
WindSway	Vertex sway for foliage and cloth props	`uTime`, `uStrength`	Wisteria, grass, banners (Acts 2, 7)	Fewer instances
RingRipple	Up to 3 ring waves on a surface	`uImpacts[3]`	Tea cup, Act 2	Same
ElementBurst	Huge element shape (dragon, tiger) revealed by dissolve	`uReveal`, `uColor`, shape mesh or texture	Pillar strikes, Act 3	Smaller texture
ScreenSplit (post)	Splits the screen along the swipe line	`uLineStart`, `uLineEnd`, `uOffset`	Act 4	Same
SpeedLines (post)	Streaks from the edges during fast motion	`uSpeed`	Act 1 fall, whip pans	Off

Effect	What it is	Main uniforms / inputs	Used in	Lite tier
Bloom + Vignette (post)	Glow on lanterns and elements, darker edges	intensity, threshold	Everywhere	Half-resolution bloom
SunriseGrade + Eyelid (post)	Night-to-dawn color grade, white-out, eyelid close/open	`uDawn`, `uWhite`, `uLid`	Acts 5 to 7	Same

Rules for every shader:

- Write shaders as exported template strings in `.glsl.ts` files (no extra build config needed).
- Build custom materials by extending three.js materials (`onBeforeCompile` or a shared shader chunk), so skinning, morph targets and instancing keep working.
- One material per effect type, shared across all instances; per-instance differences go in instance attributes.
- Animate only by changing uniforms inside `useActFrame` using the scaled time, so slow motion works automatically.
- Merge all post effects into the single `EffectComposer`; ScreenSplit and SpeedLines are disabled (not just set to zero) when not in use.

Asset pipeline

Every page starts as a complete, approved Blender scene — characters, materials, lighting, cameras and every shot — and only then becomes a web page; the browser plays what Blender already shows. All assets flow `blender/` → `raw-assets/` → `scripts/optimize-assets.mjs` → `public/`, and nothing reaches `public/` unoptimized.

The Blender-first workflow (repeat per act)

1. **Block out** the act in Blender from the screenplay: grey-box layout and one animated camera, with a timeline marker per shot id.

2. **Build assets:** reuse library kit pieces, add new props, and bring in the characters this act needs.
3. **Shade:** apply the shared toon material and outline from `blender/library/`; bake environment lighting with Cycles.
4. **Rig and animate:** characters on the shared Rigify layout; one action per move, named `<character>_<move>`.
5. **Lay out the film:** animate the camera for every shot, check timing against the screenplay's runtimes.
6. **Preview and approve:** render the whole act in Eevee to `blender/renders/act<N>.mp4` and compare it shot by shot with the screenplay. Only an approved act moves on.
7. **Export:** `export_glb.py` (models and clips), `export_shots.py` (camera), `bake_vat.py` (crowds), then `pnpm assets` to optimize.
8. **Build the web page** for that act (see Build plan).

Claude Code + Blender

- **Live:** a Blender MCP server lets Claude Code create and adjust objects, materials, rigs and keyframes in your open Blender session while you watch and give feedback.
- **Reproducible:** every change made live is also saved as a `bpy` script in `blender/scripts/`, so any scene can be rebuilt from scratch with `blender --background --python blender/scripts/build_act1.py`. If a `.blend` breaks, nothing is lost.
- **Where Claude Code is strongest:** architecture, kits, props, layout, materials, rigging setup, camera work, batch export and optimization. Expressive anime faces and bodies come out best when Claude Code starts from a sculpted or base character mesh and then refines, rigs, shades and animates it. Plan extra time for the first hero character; the Pillars reuse its rig and much of its body.

Characters

Character	On screen at once	Triangles (LOD0 / LOD1)	Bones	Textures	Notes
Swordsman (hero)	1	25k / 8k	⩽ 65	1 × 2048 atlas	Face shape keys for blink, inhale, battle cry
Pillars	up to 4	18k / 6k each	⩽ 65	1 × 1024 each	Shared rig and base animation set

Character	On screen at once	Triangles (LOD0 / LOD1)	Bones	Textures	Notes
Upper Rank demons	1–2	20k / 7k	⩽ 80	1 × 1024	Extra limbs as extra bone chains
Demon King	1	30k / 10k	⩽ 100	1 × 2048	Forms via shape keys + toggled parts
Weak demons	up to 200	1.5k, one LOD	none (VAT)	Shared 512	1–3 baked crowd clips
Crow	1	3k	⩽ 20	512	`land`, `pick_up`, `fly`
Koi	6–8	800	none (VAT)	Shared 256	One swim loop

Animation rules: 30 fps keys; root motion removed (code moves characters along paths); loops marked with a `_loop` suffix; cloth and hair simulated in Blender and baked into bones or shape keys, never simulated in the browser.

Environments

- Real-world scale (1 unit = 1 meter), origin at the base, transforms applied before export.
- Built from kits: model a corridor kit, not a corridor. Repeated pieces export once and are instanced in code.
- Lighting baked with Cycles into the color texture; scenes use 1–3 real lights (mostly for characters).
- Large pieces get a LOD1 made with Blender's Decimate modifier.

Export settings and formats

Asset type	Exported as	Optimized to	Size rule
Characters	GLB with skin, shape keys, all actions as clips	meshopt + KTX2 (UASTC for faces, ETC1S for clothes)	Per the Characters table
Environments and props	GLB per collection, modifiers applied	meshopt + KTX2 (ETC1S)	Max 2048 px textures, prefer 1024

Asset type	Exported as	Optimized to	Size rule
Crowd animations (VAT)	Position + normal textures from `bake_vat.py`	Kept as half-float EXR or 16-bit PNG, **not** KTX2 (needs precision)	Max 512 × 512 per clip set
Camera	`shots/<act>.json` from `export_shots.py`	—	One keyframe per Blender key, not per frame
Effect textures (noise, stroke order, element shapes)	PNG	KTX2 or small PNG	512 px max
HUD icons (mute, menu)	SVG	SVGO	Under 5 KB each
Sound effects	WAV	One audio sprite: WebM (Opus) + MP3 fallback	Under 600 KB total
Music and ambience	WAV	WebM (Opus, 96 kbps) + MP3 fallback	Streamed, never preloaded whole
Project preview videos	Screen recordings	WebM + MP4, 720p, 6–10 s loops, muted	Under 1.5 MB each, loaded when the talisman opens
Fonts	Open-license brush display font + clean body font	`next/font`, Latin subset	Under 100 KB total

```
# inside scripts/optimize-assets.mjs (needs KTX-Software installed)
gltf-transform optimize raw-assets/models/swordsman.glb
public/models/swordsman.glb \
  --compress meshopt --texture-compress ktx2
```

Art sourcing

All characters, demons and environments must be original designs inspired by the anime's style, not copied, with music and sound effects from royalty-free sources whose licenses are saved in `raw-assets/LICENSES.md`. Free CC0 sources: Poly Haven (textures, HDRIs, props), Kenney (props), Freesound (filter by CC0).

Performance

Full 3D characters are affordable because only a few are ever skinned and animated at full detail at once, crowds are VAT, lighting is baked, and every scene streams in one act ahead. A feature that breaks a budget is simplified in Blender (fewer triangles, smaller textures, a LOD) before it is cut.

Budgets per quality tier

Setting	High	Medium	Lite
Max pixel ratio (DPR)	2	1.5	1
Triangles on screen	1.2M	700k	300k
Draw calls per frame	200	140	80
Skinned characters animating at LOD0	6	4	2 (others at LOD1)
GPU texture memory	400 MB	256 MB	128 MB
VAT crowd instances	200	120	60
Particles per system	20k	10k	4k
Lanterns (Act 1)	300	200	120
Real-time shadows	1 shadow light, 1024 map, characters only	Blob shadows under characters	Blob shadows
Outlines	All characters + key props	Characters	Hero + current enemy
Antialiasing	MSAA 4×	SMAA	None
Bloom	Full resolution	Half	Quarter

Setting	High	Medium	Lite
Speed lines, motion blur	On	On	Off
Frame cap	None	None	30 fps (saves battery, avoids heat throttling)

Rules Claude Code must follow

1. **No allocations in `useFrame`.** Create vectors, colors and matrices once and reuse them.
2. **Instance anything repeated more than 5 times** (lanterns, kit pieces, debris, foliage, doors); crowds use VAT, never many skinned meshes.
3. **LOD every character and large prop** with drei `<Detailed>`; switch distances are set per tier.
4. **Pause animation mixers** of characters that are off-screen or fully hidden; update visible ones with the scaled delta.
5. **Compile and warm up before showing.** While an act loads, call `gl.compile(scene, camera)` and render one hidden frame of each skinned character, so the first frame of an act never stutters.
6. **Code-split each act.** Act code loads with dynamic `import()` together with its assets; the first download is only Act 0 and the core systems.
7. **Dispose on unmount:** geometries, materials, textures, skeletons, mixers and render targets of the previous act.
8. **Bake, don't simulate:** lighting, cloth and hair are baked in Blender; no physics engine in the browser.
9. **Limit transparency:** foliage uses `alphaTest`, not blending; transparent effects are kept small on screen.
10. **React stays out of the loop.** Per-frame values never go through `useState`.
11. **Pause when hidden:** stop the render loop when the tab is hidden or the recruiter page is open.
12. **Measure every act:** r3f-perf overlay (FPS, draw calls, triangles, textures), Chrome Performance panel for long frames, and a real mid-range Android phone through remote debugging. An act isn't done until it meets its tier budgets.

Accessibility, fallbacks and SEO

Every visitor gets the full portfolio content, whatever their device, settings or patience; the 3D film is the enhanced layer on top of a plain, fast page.

Recruiter page (`/cv`)

- Statically generated from the same `content/` files: name, role, about, skills, projects, experience, contact form.
- No WebGL, no animation library; loads in under 1 second.
- Linked from the loading screen ("Skip the story"), the quick menu, and the end card.
- This is also the page search engines index: proper `<title>`, meta description, Open Graph image (a still frame from the film), and a `Person` JSON-LD block.

Automatic fallbacks

Situation	What happens
No WebGL support, or WebGL context lost twice	Redirect to `/cv` with a short note
System setting "reduce motion" is on	Camera flights become 0.6 s crossfades, no shake, no speed lines, slow motion kept short; a toggle lets visitors turn the full film back on
Very low GPU score or low memory phone	Lite tier, and the loading screen suggests `/cv` as an option
Save-Data header or slow connection	Loading screen suggests `/cv` before downloading Act 1

Accessibility checklist

- [ ] Every interaction has a keyboard path (see the Input table in Core systems).
- [ ] Visible focus outlines on all DOM controls.
- [ ] Mute button reachable from the first screen; no sound before the Enter click.
- [ ] All readable text is real HTML with sufficient contrast over the scene (use a soft dark backdrop behind overlays).
- [ ] A live region announces each new act name for screen readers.

- [ ] No flashing faster than 3 times per second (the thunder and impact flashes must respect this).
- [ ] The canvas has `aria-hidden="true"`; content lives in the DOM overlays.

Mobile

- Portrait layout: camera fov widens and key framing shifts so subjects stay centered.
- Swipe gates use the same swipe gesture; hold gates use press-and-hold on a large button.
- Touch scrolling uses Lenis with touch smoothing off, so it feels native.

Contact backend and security

The crow's message is a POST to `/api/contact`, a Next.js route handler that validates the input, blocks spam, and emails it through Resend; the API key never reaches the browser.

Request flow:

1. The paper form sends `{ name, email, message, website }` as JSON. `website` is a hidden honeypot field; real people leave it empty.
2. The route validates with the shared zod schema: name 1–80 characters, valid email, message 10–2000 characters.
3. Spam checks: honeypot must be empty; reject if the form was submitted less than 3 seconds after it appeared; limit each IP to 5 messages per hour.
4. Send the email with Resend to the owner's inbox, with the visitor's email as reply-to.
5. Return `{ ok: true }`. The crow animation starts immediately on submit, but if the request fails, the crow returns and the paper shows a friendly error with the plain email address as a backup.

Security rules:

- `RESEND_API_KEY` and `CONTACT_TO_EMAIL` live only in Vercel environment variables and a local `.env.local` that is git-ignored.
- Never render visitor input as HTML; escape it in the email template.
- Set security headers in `next.config`: Content-Security-Policy allowing only own assets (plus `blob:` for workers and textures), `X-Content-Type-Options`, `Referrer-Policy`.

- For version 1, rate limiting can be in-memory per server instance; move to a hosted store (for example Upstash Redis) if spam appears.

Testing and deployment

Every push runs type checks, unit tests, end-to-end tests and a Lighthouse check, and every branch gets its own Vercel preview link to test on a real phone.

Test layer	Tool	What it checks
Types and lint	`tsc --noEmit`, ESLint	Strict TypeScript, no unused code, React hooks rules
Content	Vitest + zod	All `content/` files pass their schemas and cross-references
Shot data	Vitest	Every shot has keys sorted by `t` from 0 to 1, valid cue names, gates that exist
Systems	Vitest	Director act order, gate resolution, time-scale ramps, swipe detection math
End to end	Playwright	Loading → Enter → jump to each act via quick menu → screenshot each act's start; contact form submits (API mocked); `/cv` renders all content
Fallbacks	Playwright	Reduced-motion mode, WebGL disabled → redirect to `/cv`
Performance	Lighthouse CI	`/cv` scores 95+; home page loading screen under 1.5 MB

Deployment:

1. GitHub repository connected to Vercel.
2. `main` branch = production; every other branch = preview deployment.
3. GitHub Actions runs the checks above; a failing check blocks merging into `main`.
4. Assets in `public/` get long cache headers (`Cache-Control: public, max-age=31536000, immutable`) with hashed or versioned file names, so returning visitors download almost nothing.

5. Before each release: one full playthrough on a desktop browser, one on a mid-range Android phone, one on an iPhone (Safari).

Build plan for Claude Code

Each act is built in two phases: first the complete scene in Blender (characters, materials, lighting, camera, every shot, approved as a preview render), then the web page that plays it. Before starting, download this document and the screenplay as Markdown into the repository as `docs/tdd.md` and `docs/screenplay.md`, and add the `CLAUDE.md` below at the root.

CLAUDE.md (copy into the repository root)

```
# Midnight to Dawn — 3D portfolio

## What this is
Scroll-driven 3D portfolio told as one continuous story in 8 acts.
Every act is built and approved in Blender first, then played on the web.
- Story and shots: docs/screenplay.md
- Architecture, budgets, rules: docs/tdd.md (source of truth)
- Progress log: docs/progress.md

## Commands
pnpm dev | pnpm build | pnpm typecheck | pnpm lint
pnpm test | pnpm test:e2e
pnpm blender:export <act>   # runs export_glb.py, export_shots.py,
bake_vat.py headless
pnpm assets                 # optimize raw-assets into public

## Blender rules
- All 3D starts in blender/. Never hand-edit GLBs or camera keys in public/
or shots/.
- Every change made live through the Blender MCP must also be saved in a
  bpy script in blender/scripts/ so the scene can be rebuilt from scratch.
- Shared toon material, outline and kit pieces come from blender/library/.
- Names: collections act<N>_<thing>, actions <character>_<move>,
  camera markers = screenplay shot ids (e.g. 1.7-name-reveal).
- Stay within the Characters table budgets (triangles, bones, textures).
- An act is only exported after its preview render is approved.

## Web rules
```

```
- Follow docs/tdd.md. Ask before adding any dependency.
- TypeScript strict, no `any`.
- Portfolio text only in content/. No act imports another act.
- Inside useFrame: no allocations, no setState; read the store with
  useStore.getState(); animate with useActFrame so slow motion works.
- Readable text is HTML placed on 3D surfaces with <Html transform occlude>.
- Instance repeated objects, VAT for crowds, LOD for characters, dispose on
unmount.
- Stay within the quality tier budgets in docs/tdd.md > Performance.
- Every interaction needs a keyboard path.
- Before finishing a task: run typecheck, lint and tests, and fix failures.
- At the end of each task, update docs/progress.md (done, next, open issues).
```

Milestones

#	Milestone	Blender phase is done when	Web phase is done when
M0	Setup	Blender MCP connected to Claude Code; `blender/` folders, Git LFS, export scripts run headless on an empty test scene	Next.js + TS strict + Tailwind + stack pinned; CI green; preview deploy on Vercel
M1	Content + recruiter page	—	`content/` types, schemas, sample data; `/cv` shows everything; Lighthouse 95+
M2	Core systems	A test scene with a moving camera and markers exports to valid shot JSON	Director, shot playback, camera rig, time scale, Lenis, beats, hold and swipe gates, quick menu, skip; 60 fps with test scene
M3	Shared library + hero	Toon material, outline, lantern and door kit; swordsman modeled, rigged, with Act 1 clips; turntable render approved	Swordsman GLB plays all clips in a test page and looks the same as in Blender (toon + outline match)

#	Milestone	Blender phase is done when	Web phase is done when
M4	Act 0 + Act 1	Full shaft kit, door tunnel, name stroke-order texture, camera for shots 1.1–1.9; preview render approved	Loading scene, audio unlock, infinite shaft, door name reveal, landing; within budgets
M5	Act 2	Baked room, rigged scrolls, tea set, tatami flip, camera; render approved	Scroll text on surfaces, read gates, ripple, floor transition
M6	Act 7 + contact	Baked mansion and garden, crow and koi, camera; render approved	Paper form on 3D paper, real email received; **story playable end to end** (Acts 3–6 as placeholders)
M7	Act 4	6 Upper Rank demons rigged and animated, balconies, power presets, camera; render approved	Beats from `projects.ts`, swipe slash, talisman cards with video and links
M8	Act 3	Pillars on the shared rig with strike sets, platforms, VAT crowd baked, camera; render approved	Pillar template driven by `skills.ts`, beacons, ash-to-icon, ascent
M9	Act 5 + Act 6	King with all forms, arena, debris kit, light cracks, camera; render approved	Phases from `experience.ts`, 3D health scroll, final hold, sunrise, dawn blink
M10	Sound, tiers, mobile, accessibility	LOD1 for every character and large prop	Full sound pass; High/Medium/Lite budgets met; reduced motion, fallbacks, keyboard paths
M11	Polish and launch	Final render pass for the Open Graph image	Real-device playthroughs, Lighthouse CI green, SEO, production deploy

How to run each session

1. **One phase per session.** A Blender session and a web session for the same act are separate, so each starts with a clean plan.

2. **Plan first.** Start Claude Code in plan mode with a prompt like the ones below and review the plan before anything is built.
3. **Watch the Blender build live,** give feedback, and make sure the final state is saved as a script.
4. **Approve the preview render** against the screenplay before exporting. Fixing a shot in Blender is far cheaper than fixing it in code.
5. **Verify the web phase:** typecheck, lint, tests, Playwright screenshots of each shot next to the Blender render, and the r3f-perf overlay against the tier budgets.
6. **Commit and log:** one commit per phase; Claude Code updates `docs/progress.md`.

Example Blender session prompt:

```
Read CLAUDE.md, docs/tdd.md (Asset pipeline, Scene specs > Act 1,
Performance)
and docs/screenplay.md (Act 1). We are on milestone M4, Blender phase.
Propose a plan: kit pieces to reuse or build, the name stroke-order texture,
camera keys and markers for shots 1.1–1.9, and the build_act1.py script
structure. Use the swordsman and library from M3. Don't build until I
approve.
```

Example web session prompt:

```
Read CLAUDE.md, docs/tdd.md (Core systems, Scene specs > Act 1, Shaders and
VFX,
Performance) and docs/screenplay.md (Act 1). We are on milestone M4, web
phase.
The approved exports are in public/models and shots/act1-fall.json.
Propose a plan: components, which VFX from the catalog, events and gates to
add
to the shot JSON, and how you'll verify each shot against
blender/renders/act1.mp4. Don't write code until I approve the plan.
```